

Deep learning in computer vision - HW#2

Tomáš Belák

December 15, 2025

Contents

1	Introduction	2
2	Stanford Dogs Dataset	2
3	RetinaNet	2
3.1	How RetinaNet finds the position of objects	3
3.2	How the loss is computed during training	3
3.3	How RetinaNet solves multiple boxes for one object	4
3.4	Class imbalance problem	4
4	Training and Evaluation	4
4.1	Dataset preparation	4
4.2	Model setup	5
4.3	Training and validation	5
4.4	Final Test Evaluation	8
5	Conclusion	8

1 Introduction

The aim of this work was to train and evaluate an object detector on a smaller real-world dataset and to learn the basic workflow of modern detection methods. For this purpose I selected RetinaNet and applied it to the Stanford Dogs dataset, which contains images of 120 dog breeds with bounding box annotations. I focused mainly on understanding how the detector works and how to prepare the data. After setting up the pipeline, I trained the model, monitored the validation performance and finally selected the best checkpoint based on AP50 and evaluated on the test dataset.

2 Stanford Dogs Dataset

For this homework I used the Stanford Dogs dataset . It contains images of 120 different dog breeds, collected from the ImageNet dataset. The images are real photos taken in many different conditions, usually containing one dog per image. Each image also comes with a bounding box annotation that marks the location of the dog. The dataset is relatively small for object detection, and some breeds have fewer examples than others, which makes the task a bit harder. The official split provides separate training and test lists and I created an additional validation set from the training part.



Figure 1: Example image of a Vizsla from the Stanford Dogs Dataset.

3 RetinaNet

RetinaNet was introduced in the paper *Focal Loss for Dense Object Detection* (Lin et al., 2018)[1] and this entire section is based on that article as well. The main idea of the paper is a new loss function called **focal loss**, which is

a modification of normal cross-entropy loss balanced with parameter α and with an extra term $(1 - p_t)^\gamma$. Here p_t is the model probability for the correct class so if the ground truth label is 1, then $p_t = p$ and if the ground truth label is 0, then $p_t = 1 - p$, where p is the predicted probability for class 1. That means that p_t is always the probability of the true label. The term $(1 - p_t)^\gamma$ reduces the influence of easy examples (where p_t is close to 1), so the model can focus more on hard and important samples. The paper shows that this helps one-stage detectors to reach higher accuracy.

3.1 How RetinaNet finds the position of objects

RetinaNet uses a structure called Feature Pyramid Network (FPN). It creates several feature maps with different spatial scales, called $P3$ to $P7$. Here $P3$ is the highest resolution map where every pixel roughly corresponds to a small region in the input image and $P7$ is the lowest resolution map where one pixel covers a much larger region. In practice it is expected that small objects are detected mostly on $P3$ or $P4$ and large objects on $P6$ or $P7$.

On every location of these feature maps we have predefined anchor boxes with different sizes and aspect ratios. On top of each FPN level RetinaNet uses two small fully convolutional subnetworks, a classification subnet and a box regression subnet, which share their weights across all pyramid levels. For each anchor the network predicts a classification score and also a regression offset that moves the anchor to the final bounding box. So the position of objects is found by matching these anchors to real objects on the different pyramid levels.

3.2 How the loss is computed during training

During training every anchor is matched to ground truth bounding boxes. If IoU between an anchor and some ground truth box is at least 0.5, then the anchor is a positive sample. If the IoU is smaller than 0.4, the anchor is negative (background). Anchors between 0.4 and 0.5 are ignored.

The classification part uses **focal loss**:

$$FL(p_t) = -\alpha_t(1 - p_t)^\gamma \log(p_t).$$

This reduces the loss from easy negatives which normally dominate the gradient. For positive anchors the network also predicts box regression offsets, and for these a smooth L1 loss is used. The final loss for each image is the sum of classification loss over all anchors and the regression loss for positive anchors.

3.3 How RetinaNet solves multiple boxes for one object

This problem is solved in two steps. In training each ground truth object is assigned to at most one anchor (the one with highest IoU). So the model does not learn to produce many boxes for one object.

During inference the model outputs many predicted boxes. To remove duplicates, RetinaNet uses non-maximum suppression (NMS). NMS takes boxes with high overlap and keeps only the one with the best confidence score. This way every object should have only one final bounding box. I believe NMS is standard among some other object detectors as well.

3.4 Class imbalance problem

One-stage detectors have a big problem with class imbalance. In this context, class imbalance means that there are many more negative examples (background anchors) than positive examples (anchors that actually contain an object of any class). On a single image RetinaNet evaluates tens or even hundreds of thousands of anchors, but only a very small fraction of them overlaps a ground truth object. If we use normal cross-entropy, the loss from background anchors dominates and the model mainly learns to predict background everywhere.

RetinaNet solves this with focal loss. The factor $(1 - p_t)^\gamma$ makes the contribution of easy negative anchors almost zero, because for them p_t is already close to 1. In other words, the detector does not waste gradient on anchors that are clearly background. The parameter α is also used to balance positive and negative examples.

4 Training and Evaluation

4.1 Dataset preparation

First I loaded the official train and test split from the files `train_list.mat` and `test_list.mat`. These files contain lists of image paths, annotation XML files and class labels. I used a helper function that reads these values and returns them in Python lists.

Then I created the class mapping. Each dog breed in Stanford Dogs corresponds to one synset (folder name). I collected all synsets from both train and test, sorted them and created a list `class_names` where index 0 is background and indices 1-120 are the dog breeds. This mapping is used to

convert folder name into a class id. The detector therefore has 121 classes in total.

From the train split I further created a validation split. I used an 80/20 division with stratification, so all breeds appear also in the validation set with same representation.

To load data I used a function that loads an image with PIL and parses the annotation XML. For each object the code reads the bounding box values `xmin`, `ymin`, `xmax`, `ymax` and saves them into a tensor. The class label is read from the folder name and converted with the class mapping. Extra fields like `image_id`, `area` and `iscrowd` are also prepared, because they are required by the PyTorch detection API.

4.2 Model setup

I used the implementation `retinanet_resnet50_fpn` from `torchvision`. The backbone has pretrained ImageNet weights, but the detection head is initialized from scratch. Both backbone and head were trained. I set the number of classes to 121 including background.

I trained on Google colab with T4 GPU runtime and when using default image size built in the RetinaNet I could only fit 2 samples per batch on to the GPU. So instead of the default 800-1333 resolution, I used a smaller range 400-667 pixels. This probably negatively affected the ability of the model.

4.3 Training and validation

For optimization I used SGD with learning rate 0.001, momentum 0.9 and weight decay 0.0005 (recommended by ChatGPT). I trained for 33 epochs with batch size 12.

During an epoch I first shuffle the training indices. Then I load each batch with images and targets. The RetinaNet model in training mode returns a dictionary of losses: mainly the focal loss for classification and the smooth L1 loss for bounding box regression. I sum these values to get the total loss.

Before the backward pass I check if the loss is finite. If it is NaN or Inf, I skip that batch. On recommendation of ChatGPT, I also applied gradient clipping with max norm 1.0 to prevent exploding gradients destroy training progress.

Figure 2 shows the training loss over the whole training process. The loss generally decreases which suggests that the model is learning representations even with smaller resolution. There is no strong sign of oscillation so I guess the chosen settings were reasonable.

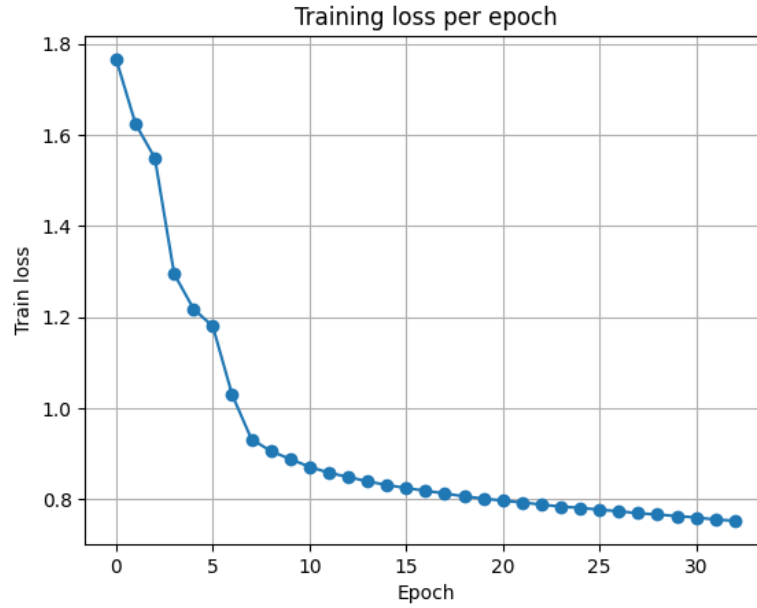


Figure 2: Training loss per epoch

After each epoch I evaluated the model on the validation set using mean average precision metric from `torchmetrics`. The metric returns the overall COCO style mAP across IoU thresholds (0.5 to 0.95), as well as the AP50 (mAP50).

Figure 3 shows the AP50 curve over the epochs. We can see that the metric's rise was quite stable however after 33 epochs it rose only to about 0.019 wich was also the best validation reached.

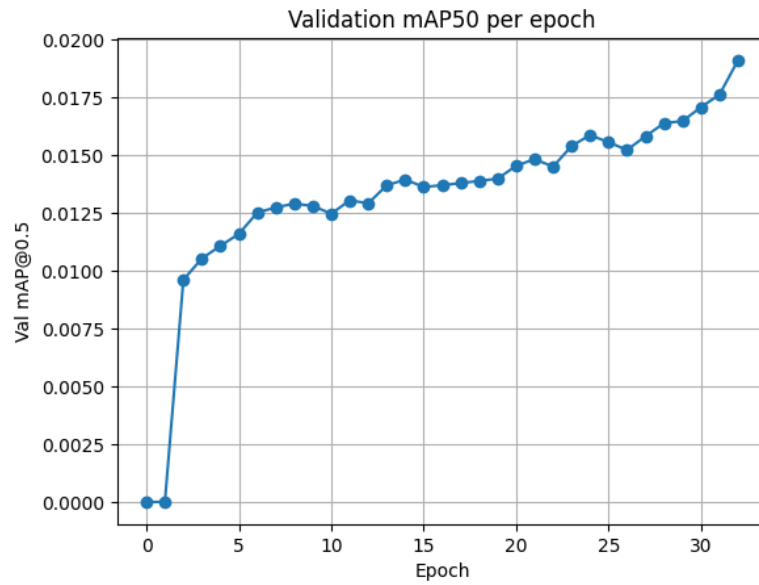


Figure 3: AP50 evolution on validation set during training

Figure 4 shows both the evolution of train loss and AP50 on validation set.

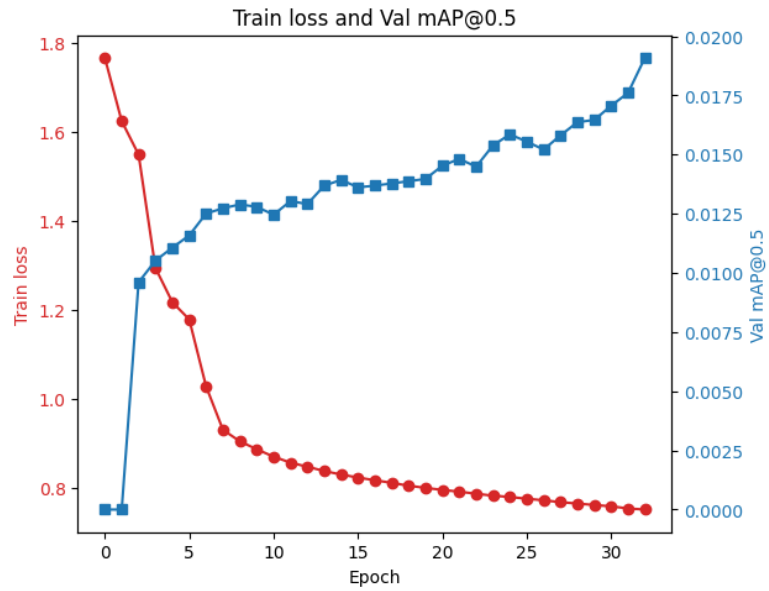


Figure 4: Training loss and validation AP50 evolution during training

4.4 Final Test Evaluation

I evaluated the final model on the test split. The same metrics were computed: overall mAP (0.5:0.95) and AP50. The results are pretty poor:

- mAP (0.5:0.95): 0.0062
- AP50: 0.0104

5 Conclusion

The low AP is probably caused by several things. The input resolution was significantly reduced due to GPU memory limits which possibly made the localization much harder. The dataset is also quite small and some breeds look very similar, so the model may not have enough variety to generalize well. With higher resolution, augmentation and longer training the performance would probably improve.

References

- [1] Tsung-Yi Lin et al. “Focal Loss for Dense Object Detection”. In: *CoRR* abs/1708.02002 (2017). arXiv: 1708.02002. URL: <http://arxiv.org/abs/1708.02002>.